



DOCUMENTAÇÃO TÉCNICA

Transmission Portal

Funcionamento, integrações e guia de implantação

Portal interno do **Setor de Engenharia — Transmissão** da Tely, que concentra as ferramentas de operação da rede de transmissão óptica: acompanhamento de demandas e RMAs, documentação dos lambdas da rede DWDM, catálogo do almoxarifado, geração de LOA e RFO e o monitoramento dos circuitos da Amazon Leo integrado ao Zabbix.

Item	Informação
Sistema	Transmission Portal
Empresa	Tely — Internet de Fibra
Setor responsável	Engenharia — Transmissão (Network Engineering — Transmission)
Desenvolvido por	Antony Araújo — Engenharia de Transmissão
Documento	Documentação técnica e guia de implantação
Versão / data	1.0 — outubro de 2026
Repositório	github.com/antonyllz/transmission-portal (branch main)
Classificação	Uso interno

Conteúdo

1. Visão geral
2. Arquitetura
3. Estrutura de arquivos
4. Dados e API
5. Páginas e funcionalidades
6. Integração com o Zabbix
7. Dashboard Amazon Leo
8. Instalação com Docker
9. Migração da VPS atual
10. Operação e manutenção
11. Segurança e observações
12. Responsabilidade e contato



1. Visão geral

O Transmission Portal é o portal interno do Setor de Engenharia — Transmissão (Network Engineering — Transmission) da Tely, desenvolvido por Antony Araújo. Reúne ferramentas do dia a dia: quadro de demandas, controle de RMAs, mapa DWDM com os lambdas em uso, catálogo do almoxarifado, geradores de LOA e RFO, timeline de casos e a dashboard dos circuitos da Amazon Leo alimentada pelo Zabbix.

É uma aplicação web leve: front-end em HTML/CSS/JavaScript puro e um back-end em Node.js **sem nenhuma dependência externa** (não usa npm install). Os dados ficam em arquivos JSON. Todos os usuários veem os mesmos dados.

2. Arquitetura

São dois processos Node.js:

Processo	Arquivo	Porta	Função
Servidor público	server/public-server.js	5454 (0.0.0.0)	Serve os arquivos estáticos do site e repassa as chamadas /api/* para a API interna.
API de dados	server/server.js	5455 (127.0.0.1)	Lê e grava as coleções JSON em server/data/, recebe o webhook do Zabbix e entrega o feed da dashboard.

```
Navegador --(http :5454)--> public-server.js
                        | /api/*
                        v
Zabbix --- (POST /api/zabbix/webhook + token) ---> server.js (127.0.0.1:5455)
                        |
                        v
                        server/data/*.json
```

O navegador mantém uma cópia local (localStorage) para abrir rápido e sincroniza com o servidor a cada 8 segundos. A gravação é feita substituindo a coleção inteira (PUT); a última gravação vence. A navegação usa endereços com # (ex.: #/rmas), então voltar/avançar do navegador e links compartilhados funcionam.

Bibliotecas carregadas por CDN no navegador (precisam de acesso à internet a partir do PC do usuário): Leaflet (mapa), leaflet-polylineoffset, SheetJS (só ao importar planilha), fontes Inter (Google Fonts) e tiles do OpenStreetMap.



3. Estrutura de arquivos

Caminho	Conteúdo
index.html	Página única com todas as telas do portal.
assets/css/*.css	Estilos: base, home, demandas, RMAs, lambdas, almoxarifado, dashboard Amazon Leo, tema escuro (profile.css).
assets/js/app.js	Inicialização e atualização periódica das telas.
assets/js/database.js	Cache local + sincronização com a API (GET/PUT).
assets/js/navigation.js	Navegação e rotas (#/...). Toda página nova precisa entrar em PAGE_ROUTES e ROUTE_OPENERS.
assets/js/home.js	Animações da home (malha óptica do topo, cards).
assets/js/demands.js	Painel de demandas.
assets/js/rma.js	RMAs.
assets/js/lambdas.js, osa.js	Mapa DWDM, canais, espectro (OSA).
assets/js/network-data.js	Topologia DWDM fixa: sites (coordenadas, função ROADM/Terminal/OLA) e trechos.
assets/js/network-routes.js	Traçado dos trechos pelas rodovias (gerado por tools/build-routes.js).
assets/js/alm.js	Almoxarifado (busca de itens e importação da planilha).
assets/js/leo-dash.js	Dashboard Amazon Leo.
assets/js/timeline.js	Case Timeline (Amazon Leo).
loa/, rfo/, rfo-generic/	Geradores de LOA e RFO (páginas independentes abertas dentro do portal).
server/server.js	API de dados.
server/public-server.js	Servidor público (estático + proxy da API).
server/zabbix.js	Webhook do Zabbix, métricas Rx e feed da dashboard.
server/data/	Dados (JSON) e token do Zabbix. NÃO vai para o Git.
tools/build-routes.js	Recalcula o traçado rodoviário dos trechos (OSRM).
Dockerfile, docker-compose.yml, docker/start.js	Empacotamento em container.
ecosystem.config.js	Configuração do PM2 (usada na VPS atual, não é necessária no Docker).



4. Dados e API

Cada coleção é um arquivo JSON em `server/data/` contendo uma lista. A gravação é atômica (arquivo temporário + rename).

Coleção	Arquivo	Conteúdo
cases	cases.json	Casos do Case Timeline.
demands	demands.json	Demandas da home (prioridade, prazo, comentários).
rmas	rmas.json	RMAs.
lambdas	lambdas.json	Canais DWDM (frequência, grid, cliente, pontas, rota).
netpos	netpos.json	Ajustes de posição/função de sites feitos no mapa.
itens / itensmeta	itens.json / itensmeta.json	Catálogo do almoxarifado e dados da última importação.
leoalarms	leoalarms.json	Eventos do Zabbix (Amazon Leo), até 3.000 mais recentes.
leozbxmeta	leozbxmeta.json	Controle da integração (primeiro/último evento).
leometrics	leometrics.json	Potência Rx por circuito (último valor + 24 h).
—	zabbix-token.txt	Token do webhook do Zabbix (gerado automaticamente).

Endpoints

Método e caminho	Uso
GET /api/<coleção>	Lê a coleção inteira.
PUT /api/<coleção>	Substitui a coleção (corpo: lista JSON, até 5 MB).
POST /api/zabbix/webhook	Recebe problema/recuperação/atualização do Zabbix. Exige cabeçalho X-Portal-Token.
POST /api/zabbix/metric	Recebe potência Rx: {"circuit": "...", "rx_dbm": -1.5}. Exige X-Portal-Token.
GET /api/leo/status	Feed da dashboard Amazon Leo (somente eventos dos 4 circuitos, sem host/IP).

5. Páginas e funcionalidades

Endereço	Página	Resumo
#/	Home	Ferramentas, painel de demandas, clientes, busca rápida do almoxarifado.
#/rmas	RMAs	Cadastro e acompanhamento (status, comentários); cola o texto do fornecedor e preenche os campos.
#/lambdas	Lambdas	Mapa DWDM (Infinera, Padtec, Ciena), seções Terminal→Terminal, canais por trecho, OSA, proteção 1+1.
#/almoxarifado	Almoxarifado	Busca instantânea de códigos; botão “Atualizar planilha” importa o relatório .xlsx.
#/loa, #/rfo	LOA / RFO	Geração de documentos em PDF.



Endereço	Página	Resumo
#/amazon-leo	Amazon Leo	Dashboard, RFO Amazon Leo e Case Timeline.
#/amazon-leo/dashboard	Dashboard	SLA, Rx, incidentes e status dos circuitos SLZ501 e CPV501 (inglês).
#/starlink	Starlink	Reservado para ferramentas futuras.

Tema claro/escuro/sistema em Configurações (ícone de engrenagem). O padrão é seguir o sistema operacional.

A topologia DWDM (sites e trechos) está fixa em `assets/js/network-data.js`. Ao incluir ou alterar sites/trechos, rode `node tools/build-routes.js` (precisa de internet) para recalculer o traçado pelas rodovias.

6. Integração com o Zabbix

O Zabbix envia cada evento para o portal por um *media type* do tipo Webhook. O portal não acessa o Zabbix; apenas recebe. Basta o servidor do Zabbix alcançar o portal na porta 5454.

Configuração no Zabbix

- **Alertas** → **Tipos de mídia**: tipo Webhook “Transmission Portal” (pode ser importado do arquivo `zabbix-mediatype-transmission-portal.yaml`). Parâmetros `url`, `token` e os campos do evento (`{EVENT.ID}`, `{EVENT.VALUE}`, `{EVENT.NSEVERITY}`, `{HOST.HOST}`, `{EVENT.NAME}`, `{EVENT.TAGSJSON}`, datas e horas, `tz_offset`).
- **Usuário** com essa mídia (“Enviar para”: qualquer texto, ex. portal), todas as severidades, e permissão de leitura nos hosts da Amazon Leo.
- **Ação de trigger** filtrando os hosts/tags da Amazon Leo, com operações, operações de recuperação e de atualização enviando para esse usuário.

Script do media type:

```
var p = JSON.parse(value), req = new HttpRequest();
var url = p.url, token = p.token;
delete p.url; delete p.token;
req.addHeader('Content-Type: application/json');
req.addHeader('X-Portal-Token: ' + token);
var resp = req.post(url, JSON.stringify(p));
if (req.getStatus() !== 200) { throw 'Portal respondeu HTTP ' + req.getStatus() + ': ' + resp; }
return 'OK';
```

Como o evento é ligado ao circuito

- Procura o ID do circuito (RJOOCR964161, SPOOCR964174, 21-90090-252671, 21-90090-252668) na tag **circuit**, no host, no nome do trigger, nos dados operacionais e nas tags.
- Se não achar o circuito, procura o site (SLZ501 / CPV501); evento só com site vale para os dois circuitos daquele site.



- Macros não resolvidas (ex.: {HOST.HOST} no botão Testar) são ignoradas.
- Token: variável de ambiente ZBX_TOKEN ou, se não existir, o arquivo server/data/zabbix-token.txt (criado na primeira execução).

7. Dashboard Amazon Leo

Site	Circuito	Ponta
SLZ501 — Ocara, CE	RJOOCR964161	Equinix RJ2
SLZ501 — Ocara, CE	SPOOCR964174	Equinix SP4
CPV501 — Sanharó, PE	21-90090-252671	Equinix SP4
CPV501 — Sanharó, PE	21-90090-252668	Equinix RJ2

- **Status:** problema ativo de severidade Alta ou Desastre = *Circuit down*; Atenção ou Média = *Degraded*; Informação e Não classificada são ignorados.
- **SLA** = $1 - (\text{tempo com problema Alta/Desastre no período} \div \text{duração do período})$. Intervalos sobrepostos contam uma vez. O período começa no primeiro evento conhecido (histórico importado a partir de 07/07/2026).
- **Filtros:** site, 24h/7d/30d/90d (padrão 30d) e UTC/BRT (padrão UTC). Atualiza a cada 10 s.
- **Rx signal:** aparece quando há dados em POST /api/zabbix/metric (ainda não configurado no Zabbix).
- Os IDs, sites e pontas dos circuitos ficam em assets/js/leo-dash.js e server/zabbix.js (manter os dois iguais).

8. Instalação com Docker

Pré-requisitos na VM: Docker Engine com o plugin Compose, acesso ao repositório GitHub (ou cópia dos arquivos) e a porta 5454 liberada no firewall (para os usuários e para o servidor do Zabbix).

Primeira instalação

```
git clone git@github.com:antonyllz/transmission-portal.git
cd transmission-portal
mkdir -p data # dados persistentes (montado em /app/server/data)
docker compose up -d --build
docker compose ps # STATUS deve ficar 'healthy'
docker compose logs -f # deve mostrar as portas 5454 e 5455
```

Acesse `http://<ip-da-vm>:5454`. Para usar outra porta, altere só o lado esquerdo em ports: "5454:5454" no docker-compose.yml (e a URL do webhook no Zabbix).

Variáveis de ambiente



Variável	Padrão	Uso
PUBLIC_PORT	5454	Porta do site dentro do container.
API_PORT / PORT	5455	Porta interna da API (não exposta).
DATA_DIR	/app/server/data	Pasta dos dados (volume).
ZBX_TOKEN	(arquivo no volume)	Token do webhook; se definido, substitui o zabbix-token.txt.
TZ	UTC	Fuso do container (os dados são gravados em UTC).

Atualizar para uma nova versão

```
cd transmission-portal
git pull
docker compose up -d --build # os dados em ./data são preservados
```

Sem Docker (alternativa)

Com Node.js 18+ instalado: `node docker/start.js` na raiz do projeto (ou PM2 com `ecosystem.config.js`). Os dados ficam em `server/data/`.

9. Migração da VPS atual

Os dados atuais estão na VPS (162.35.184.190) em `~/transmission-portal/server/data/`. Para levar tudo para a VM da empresa:

```
# 1) na VPS atual: empacotar os dados
cd ~/transmission-portal/server && tar czf ~/portal-data.tgz data

# 2) copiar para a VM nova
scp usuario@162.35.184.190:~/portal-data.tgz .

# 3) na VM nova, dentro da pasta do projeto, antes do 'docker compose up'
tar xzf portal-data.tgz # cria ./data com todos os JSON e o token
docker compose up -d --build
```

- Copiando o **zabbix-token.txt** junto, o token continua o mesmo: no Zabbix basta trocar o parâmetro **url** do media type para o endereço novo.
- Para evitar perda, pare a gravação na VPS antiga (`pm2 stop transmission-portal transmission-portal-api`) logo antes do passo 1 e aponte o Zabbix para a VM nova em seguida.
- Os usuários têm um cache local por navegador; ao abrir o endereço novo, os dados vêm do servidor normalmente.



10. Operação e manutenção

Tarefa	Comando / ação
Ver logs	<code>docker compose logs -f</code>
Reiniciar	<code>docker compose restart</code>
Parar	<code>docker compose down</code> (os dados em <code>./data</code> permanecem)
Backup	<code>tar czf backup-\$(date +%F).tgz data</code> (agendar no cron, ex. diariamente)
Restaurar	<code>docker compose down</code> ; restaurar a pasta <code>data</code> ; <code>docker compose up -d</code>
Trocar o token do Zabbix	Apagar <code>data/zabbix-token.txt</code> (ou definir <code>ZBX_TOKEN</code>), reiniciar e atualizar o parâmetro token no Zabbix.
Atualizar almoxarifado	Na página Almoxarifado, "Atualizar planilha" e escolher o <code>.xlsx</code> .
Alterar topologia DWDM	Editar <code>assets/js/network-data.js</code> , rodar <code>node tools/build-routes.js</code> , <code>rebuild</code> .

11. Segurança e observações

- O portal **não tem login**: qualquer pessoa que alcance a porta consegue ler e alterar os dados. Recomenda-se deixá-lo acessível só na rede interna/VPN.
- Funciona em HTTP. Para HTTPS, coloque um proxy reverso (nginx, Traefik etc.) na frente do container.
- O webhook do Zabbix é protegido pelo token (X-Portal-Token). Guarde o token e troque-o se vaziar.
- Os dados são arquivos JSON: não há banco de dados para administrar, mas o backup da pasta `data` é essencial.
- Escritas simultâneas na mesma coleção: vale a última (uso adequado para um time pequeno).
- Os navegadores dos usuários precisam de internet para carregar fontes, mapa (OpenStreetMap) e bibliotecas via CDN.

12. Responsabilidade e contato

Item	Informação
Setor responsável	Engenharia — Transmissão
Desenvolvimento e manutenção	Antony Araújo
Código-fonte	github.com/antonyllz/transmission-portal
Alterações	Toda alteração deve ser versionada no repositório (commit + push) antes de ser publicada no servidor.

Dúvidas, ajustes ou novas funcionalidades devem ser tratados com o Setor de Engenharia — Transmissão.